applied sciences

MDPI

Article

Leveraging RAG with ACP & MCP for Adaptive Intelligent Tutoring

Horia Alexandru Modran

Faculty of Electrical Engineering and Computer Science, Transilvania University of Brasov, 500036 Brasov, Romania; horia.modran@unitbv.ro

Abstract

This paper presents a protocol-driven hybrid architecture that integrates Retrieval-Augmented Generation (RAG) with two complementary protocols—A Model Context Protocol (MCP) and an Agent Communication Protocol (ACP)—to deliver adaptive, transparent, and interoperable intelligent tutoring for higher-education STEM courses. MCP stores, fuses, and exposes session-, task- and course-level context (learning goals, prior errors, instructor flags, and policy constraints), while ACP standardizes multipart messaging and orchestration among specialized tutor agents (retrievers, context managers, pedagogical policy agents, execution tools, and generators). A Python prototype indexes curated course materials (two course corpora: a text-focused PDF and a multimodal PDF/transcript corpus) into a vector store and applies MCP-mediated re-ranking (linear fusion of semantic similarity, MCP relevance, instructor tags, and recency) before RAG prompt assembly. In a held-out evaluation (240 annotated QA pairs) and human studies (36 students, 12 instructors), MCP-aware re-ranking improved Recall@1, increased citation fidelity, reduced unsupported numerical claims, and raised human ratings for factuality and pedagogical appropriateness. Case studies demonstrate improved context continuity, scaffolded hinting under instructor policies, and useful multimodal grounding. The paper concludes that the ACP–MCP–RAG combination enables more trustworthy, auditable, and pedagogically aligned tutoring agents and outlines directions for multimodal extensions, learned re-rankers, and large-scale institutional deployment.

Keywords: retrieval-augmented generation (RAG); agent communication protocol (ACP); model context protocol (MCP); intelligent tutoring system

Academic Editor: Paolino Di Felice

Received: 11 September 2025
Revised: 23 October 2025
Accepted: 23 October 2025
Published: 26 October 2025

Citation: Modran, H.A. Leveraging RAG with ACP & MCP for Adaptive Intelligent Tutoring. *Appl. Sci.* **2025**, *15*, 11443. https://doi.org/10.3390/app152111443

1. Introduction

Large language models (LLMs) and artificial intelligence's (AI's) quick development have significantly changed the possibilities for individualized learning in higher education. In engineering and computer science programs, LLM-powered tutoring systems can support scalable and adaptive instructional techniques by offering on-demand explanations, practice problems, and instant feedback [1]. Model hallucinations, weak multi-turn context retention, insufficient domain grounding, and threats to academic integrity when generated information is not adequately validated are some of the shortcomings of existing LLM implementations in educational settings that have been extensively described [2–5].

To address several of these issues, Retrieval-Augmented Generation (RAG) has been suggested as a viable solution. RAG enhances factual grounding and lowers the frequency of unsupported assertions in model outputs by fusing a generative model that is conditional on retrieved passages with a retrieval module that retrieves curated, domain-specific

resources [6]. Particularly in knowledge-intensive tasks like problem-solving and programming support, recent RAG applications in education indicate improved answer accuracy and better alignment with course materials [7,8]. However, the existing literature still fails to adequately address two practical issues. First, modularity and reuse are hampered by the lack of established communication patterns across the many system components (retrievers, context managers, pedagogical rule engines, and generating agents), which makes multi-agent orchestration and interoperability frequently ad hoc. Second, the system's ability to maintain consistent, course-level context over time and to reason about student progress is constrained since context management across multi-turn conversations and across course sessions is usually restricted to ephemeral session information or simple memory buffers.

Aspects of these issues have been examined from a complementary standpoint in earlier studies. Potential advantages and moral dilemmas pertaining to student assistance and evaluation were described in research on using AI and generative chatbots in engineering education [9]. Through moderation measures, specialized GPT-based assistants designed for programming education have shown promise in supporting code generation, debugging, and pedagogical scaffolding while maintaining academic integrity [10]. In order to ground LLM responses in carefully chosen course materials, an intelligent tutoring agent based on RAG was later created. In pilot deployments, this agent demonstrated gains in domain relevance and decreased hallucinations. Large vision–language models and multimodal embeddings were used in more recent iterations of this RAG framework to incorporate multimodal resources (images, diagrams, and video), increasing the tutor's ability to manage a variety of educational materials [11].

Despite recent developments, the lack of explicit context-sharing techniques and protocolized agent interactions still limits scalability and transparency. While ACP can standardize message formats and orchestration for modular components, such as queries, context-update events, retrieval directives, provenance/citation metadata, and policy flags, MCP can specify how session-, task-, and course-level context are represented, fused, versioned, and expired. When combined, these protocols improve explainability and provenance tracing of generated responses, facilitate integration with institutional ecosystems (LMSs, content repositories, and assessment platforms), and allow decoupled retrievers, context managers, pedagogical-policy agents, and generators to interact predictably. Thus, protocolization facilitates intelligent tutoring implementations that are more interoperable, scalable, and auditable.

This paper presents a protocol-driven hybrid architecture that uses RAG in conjunction with (i) a Model Context Protocol (MCP) to maintain, fuse, and expose contextual state at dialog, task, and course scopes and (ii) an Agent Communication Protocol (ACP) to standardize inter-agent messaging and orchestration. The method is applied in a prototype system that uses MCP-mediated context fusion to affect retrieval and re-ranking, indexes carefully chosen educational materials into a vector store, and coordinates generation through ACP-coordinated agents that annotate responses with provenance and educational metadata. A case study in engineering and programming courses compares the method to a baseline RAG tutor in terms of user-perceived trustworthiness, pedagogical alignment, context retention, and retrieval accuracy.

The primary contributions of this work are the following: (1) definitions of ACP and MCP as complementary protocols for context-aware, interoperable tutoring agents; (2) a pipeline for retrieval and context-fusion that uses MCP signals to re-rank retrieved candidates before RAG prompting; and (3) an empirical evaluation and prototype implementation showing better retrieval accuracy, fewer hallucinations, and better alignment with course objectives in comparison to baseline RAG systems. According to the findings,

protocol-grounded RAG agents can provide higher education with more transparent, dependable, and pedagogically aligned support. They also encourage greater research on standardization and widespread implementation.

The structure of the remainder of the manuscript is as follows. In Section 2, relevant research on agent designs, RAG, and context management in instructional agents is reviewed. Materials and procedures are described in full in Section 3, along with the integration pipeline with RAG and ACP/MCP specifications. Results from the experiments and case study are presented in Section 4. Practical aspects, constraints, and ramifications are covered in Section 5, and future research prospects are outlined in Section 6.

2. Background and Related Work

By combining dense retrieval from an external knowledge source with a generative model conditioned on retrieved passages, Retrieval-Augmented Generation (RAG) has become a popular and useful method to enhance the factual grounding of large language models (LLMs). RAG architecture reduces unsupported assertions and improves conformity with curriculum objectives in educational contexts by constraining generative outputs with carefully selected course content and bibliographic resources. These features are critical for reliable intelligent tutoring. Recent empirical research shows that RAG-enhanced tutors can generate responses that learners prefer and often outperform unguided LLMs on domain-specific question answering and student support tasks, but it also points out significant trade-offs. Specifically, too rigorous grounding (i.e., making the generator rely on a single recovered fragment) can lower the pedagogical adaptability and naturalness of explanations, which can decrease perceived usefulness while enhancing factuality [12]. When RAG is integrated into pedagogical workflows, complementary classroom and pilot deployments demonstrate quantifiable learning gains. They also demonstrate that, when paired with suitable prompting and evaluation strategies, RAG can function as a dependable substrate for automated tutor assessment and feedback [13,14]. In STEM fields, where conceptual linkages are essential to sound reasoning, hybrid techniques that supplement RAG with structured knowledge representations have also been shown to improve grounding in assessment situations [15–17].

Despite these grounding advancements, robust RAG tutor deployment at scale is hampered by two persistent system-level issues. First, the lack of standardized interaction patterns results in ad hoc compositions that are challenging to extend, audit, or reuse. Realistic tutoring systems usually require a combination of specialized components, including document retrievers, candidate re-rankers, prompt construction modules, pedagogical policy engines, and generation agents. Thus, multi-agent orchestration models have been investigated recently, wherein decision, decomposition, router, and retrieval agents coordinate retrieval techniques, simultaneous searches across heterogeneous sources, and the merging or arbitrating of candidate responses [18–21]. It has been demonstrated that learning-based schedulers and hierarchical multi-agent designs can dynamically create workflows that balance accuracy per query, cost, and latency; these architectures allow for more precise control over failure modes and resource consumption and enhance QA accuracy on complex and multimodal benchmarks [22,23]. Protocol specifications similar to a Model Context Protocol (MCP) and an Agent Communication Protocol (ACP) are meant to provide the practical requirements for explicit message schemas, lifecycle semantics, and provenance reporting, as highlighted in surveys of emerging standards for agent interoperability [21]. Multi-agent orchestration can significantly lower hallucination rates and operational costs while improving the groundedness of responses under production constraints, as demonstrated by real-world agentic RAG deployments (such as in domain-specific advising and university admissions counseling) [22].

Second, there is still a problem with persistent context management over course sessions and multi-turn discussions. These techniques allow for more customized scaffolding and significantly increase verbal coherence over prolonged contacts. Complementary methods provide more pedagogically meaningful responses and enable higher-level tasks like adaptive sequencing and lesson planning by grounding retrieval in organized student models (skill trees, knowledge graphs, or learner profiles) [23]. This way, instructors have more transparent provenance and audit trails for automated recommendations when retrieval and generation operations are connected to explicit context tokens or student model nodes. Typically, off-the-shelf RAG pipelines use rolling context windows or ephemeral session buffers, which are insufficient for simulating multi-stage laboratory work, past misconceptions, and student growth. In order to tackle this issue, memory-augmented architectures and external context managers have been suggested; these solutions preserve explicit memory stores, implement relevance-based pruning, and enforce lifecycle policies (such as scope rules and time-to-live) to eliminate stale signals and preserve pedagogically relevant traces [24].

A third level of complexity is introduced by the requirement to manage multimodal instructional artifacts. In order to facilitate retrieval and grounding across heterogeneous payloads, multimodal RAG architectures that incorporate vision–language encoders and multimodal embeddings have been developed. Diagrams, laboratory images, simulation outputs, and lecture recordings are examples of modern educational resources [25]. Tutors may now answer questions that call for diagram interpretation or video frame analysis thanks to advancements in cross-modal retrieval and visual question answering made possible by bridge-style encoders and contrastive multimodal representations. Early results show significant benefits in STEM and practice-oriented domains where visual artifacts are essential to learning, but implementing multimodal RAG requires domain-specific preprocessing (like frame sampling, optical character recognition, audio transcription) and vector stores that support both embeddings and heterogeneous metadata.

In recent research, evaluation procedures for RAG-based tutoring systems have also advanced. The quality of candidate selection is measured using standard information retrieval measures (Precision@k, MRR, nDCG), while post-test learning gains, exact match/F1 for factoid QA, and survey instruments evaluating perceived usefulness and trust are used to assess downstream educational results. The results of ablation studies comparing baseline RAG, RAG enhanced with context managers, and multi-agent RAG variants show consistent improvements: multimodal grounding produces especially significant benefits on tasks requiring visual or procedural reasoning, while context fusion and agent orchestration improve retrieval precision and decrease unsupported assertions [12–15,20–22]. However, observed results differ by domain: procedural and multimodal questions demonstrate the most obvious value of hybrid solutions, whereas open-ended conceptual queries emphasize the tension between grounding faithfulness and explanatory richness.

Lastly, the literature analysis reveals a number of unresolved issues. First, there aren't many commonly used, officially defined protocols for provenance reporting, context lifecycle policies, and agent message semantics in the field; standardization is necessary for regulatory compliance, reproducibility, and integration with institutional ecosystems (LMS, assessment platforms) [19]. Second, there is still a lack of developed principled approaches to strike a balance between pedagogical adaptability and grounded integrity. While loose limits allow for hallucinations, excessive constraints on generation can reduce explanatory power [12]. Third, the greater computational and annotation costs associated with multimodal RAG make scalability more difficult and privacy issues in educational deployments more likely. Furthermore, there are few long-term studies of learning trajectories and instructor acceptability; the majority of assessments that have been published are either

short-term pilots or domain-specific deployments, which raise concerns about operational adoption and long-term learning improvements. Furthermore, dense passage retrieval approaches have proven effective for open-domain QA [26]

These threads collectively imply that RAG is a required but insufficient basis for reliable automated tutoring. RAG must be integrated with formal context protocols that represent, combine, and retire contextual signals across dialog, task, and course scopes, as well as with standardized agent communication conventions that provide modular orchestration and provenance tracking, in order to advance practice. Taking this stance, the current work suggests an ACP–MCP–RAG architecture that integrates MCP-mediated context fusion into the retrieval and re-ranking process, permits multimodal retrieval where appropriate, and formalizes inter-agent message schemas and context lifecycle policies. The protocol specifications, implementation specifics, and empirical assessments that evaluate the effects of protocolized coordination and context fusion on retrieval accuracy, context continuity, and pedagogical alignment in comparison to baseline RAG systems are provided in the following sections.

3. Materials and Methods

This section describes the prototype ACP–MCP–RAG system, datasets and preprocessing procedures, the RAG retrieval with context-fusion pipeline, the protocol specifications for inter-agent communication and context management and the implementation details for a Python reference implementation. The approaches aim to give sufficient information for evaluation and reproducibility.

3.1. System Environment and Reproducibility

The prototype and all benchmark runs were executed on commodity hardware to ensure reproducibility. The primary evaluation host (RouterAgent, MCP server and agent containers) ran on Ubuntu 22.04 LTS, provisioned with an Intel Xeon E5-2620 v4 (8 vCPUs) virtual machine, 32 GB RAM and 500 GB disk storage. Indexing and embedding experiments used a separate workstation equipped with an NVIDIA GTX 1080 Ti GPU (used only for embedding experiments; FAISS-CPU was used for the production retrieval runs reported in this manuscript). All services were containerized with Docker 20.10 and orchestrated using docker-compose 1.29.2. The software stack used in the evaluation included Python 3.13, FastAPI 0.88, Uvicorn 0.20, sentence-transformers 2.2.2, faiss-cpu 1.7.4, and crewai_tools 0.75. External LLM calls were performed via the provider's REST API (configuration and latency are noted for each experiment).

Figure 1 illustrates the deployment architecture of the intelligent tutoring system.

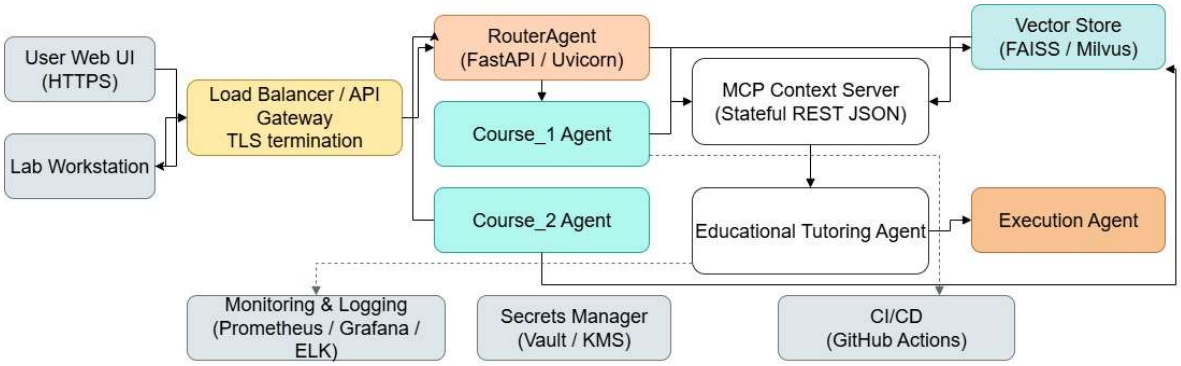

Figure 1. Deployment Diagram.

3.2. System Overview and Design Goals

This paper suggests a modular, ACP-based tutoring platform that combines an explicit Model Context Protocol (MCP) layer with retrieval-augmented generation (RAG) to generate pedagogically limited, context-aware responses for scenarios in higher education. In order to allow for the independent development, testing, and replacement of each subsystem, the implemented prototype is purposefully divided into components:

(i) An ACP client called RouterAgent that orchestrates agents in response to user queries;
(ii) Two ACP agents specific to a single PDF course file (course1 and course2) that expose RAG functionality; course1 is the text-focused PDF course book for Virtual Instrumentation [27], while course2 is the Software for Telecommunications course (multimodal: PDF, figures, and brief video segments) [28];
(iii) A JSON/HTTP-accessible lightweight MCP context server;
(iv) A Vector Store (embeddings + FAISS index) that RAG uses;
(v) An educational tutoring agent that employs pedagogical policies and provides provenance-aware answers.

For constrained code/check calculations, a streamlined Execution Agent is provided. In Figure 2, the high-level architecture is displayed.

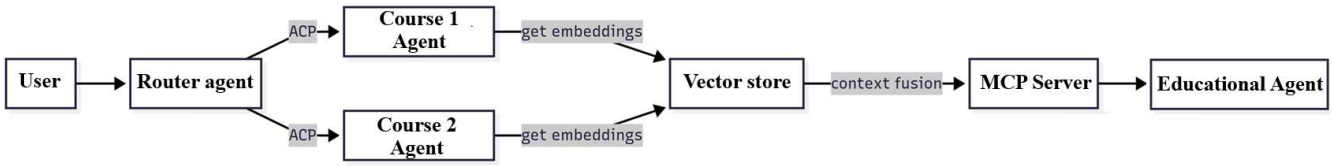

Figure 2. System Pipeline.

Two practical constraints—reproducible experimentation and interoperability between agents—guide the implementation decisions. While MCP interactions use JSON over HTTP to stay straightforward and language-neutral, inter-agent messages follow the Agent Communication Protocol (ACP) model, which is modeled using the *acp-sdk* Python library. Within each agent container, the course materials are contained as individual PDF files located at /data/course1.pdf and /data/course2.pdf.

The RagTool implementation (*RagTool* from crewai) is used for RAG indexing and retrieval. Token-based chunking parameters were chosen to balance evidence granularity and prompt/context capacity. With LLM providers offering 4k–8k token windows, a per-chunk size of 1200 tokens with overlap 200 tokens gives (i) fragments that usually contain coherent paragraphs or small sections useful for factual grounding, (ii) modest redundancy to avoid split evidence across chunk boundaries, and (iii) manageable prompt assembly for top-m = 5 passages without exceeding typical context limits. Sensitivity tests over {512, 1024, 1200, 1500} token chunk sizes and overlaps {50, 200} were performed. Where possible, token counts are computed using tiktoken tokenization to align with provider tokenization.

The runtime flow is as follows. The RouterAgent receives a query from the user interface (UI) as a JSON envelope. To determine whether to send the request to course1 (textually oriented) or course2 (multimodal), the RouterAgent employs a straightforward classifier (keyword heuristics). The RouterAgent obtains an MCP snapshot for the current session (GET */mcp/{session_id}*) before calling a course agent; the target agent receives this snapshot in the ACP message. Course agents use RagTool to execute RAG retrieval (top-K candidates), returning a provenance part encoded as a JSON MessagePart (sources, chunk ids, scores) in addition to a brief text response. Before sending the user a final response with a provenance tag, the RouterAgent optionally confers with the Educational Tutoring Agent for pedagogical checks. The conceptual organization of retrieval, MCP fusion, and

re-ranking is as follows: compute pedagogical weight (ped), compute similarity score (sim), and compute MCP context relevance (context); re-rank using a composite score. The system records MCP updates via POST */mcp/update* so ongoing sessions maintain continuity across turns.

The main objectives of the design are

- Interoperability: MCP is a distinct, HTTP-accessible service that enables non-ACP clients to interact with contextual state; agents must communicate using a standard message envelope (ACP).
- Reproducibility and modularity: Docker Compose is used to replicate the entire stack, and each agent is packaged as a separate process (FastAPI + *acp_sdk.server* or *acp_sdk.client* usage). Course PDFs are isolated, easily accessible artifacts that make replication and indexing easier.
- Explainability and provenance: To enable traceable responses in the manuscript and evaluations, each agent response contains a JSON provenance section that lists source chunk identities, document ids, and relevance scores.
- Pedagogical safety and policy control: The Educational Tutoring Agent has the authority to veto or alter RAG outputs prior to distribution and enforces instructor restrictions (such as the prohibition against giving direct answers or suggestions to exam questions).
- Scalability and extensibility: In order to accommodate bigger corpora, indexing can be done offline, and the Vector Store and RagTool backends can be plugged in (FAISS, Milvus, or cloud services).
- The architectural justification and specific parameter defaults used in the experiments are provided in this section; the ACP/MCP protocol specifics, the RAG integration, and the implementation details necessary to replicate the reported results are described in Sections 3.3–3.5.

Unlike a direct question–answer system, the Educational Tutoring Agent follows a scaffolded tutoring strategy guided by the hinting_level and policy_control parameters included in the MCP snapshot. When a student question is received, the RouterAgent forwards it to the appropriate course agent (e.g., Course 1 for Virtual Instrumentation), which retrieves the most relevant passages using the RAG component. The Educational Tutoring Agent then reformulates the evidence into a pedagogical explanation rather than a direct solution.

For example, when a student asked "How can I measure a signal's RMS value in LabVIEW?", the agent first generated a Level 2 hint response:

{"hinting_level": 2,"response": "Think about using a block that computes power-related quantities. The RMS function is located under the Signal Processing → Waveform Measurements palette."}

The system, with policy_control set to restricted, added an annotated image clip from the official course PDF in place of a complete functional solution when the same learner asked again, "Can you show me the block diagram?" This forced the students to manually link the blocks.

Similar to this, a Level 1 hint that was generated in response to a student's question concerning "SOAP message structure" in the Software for Telecommunications course encouraged investigation of the "en-velope–header–body" hierarchy prior to revealing the XML snippet. Students are guaranteed to interact with the content rather than merely copying a response thanks to this progressive feedback methodology.

These behavior patterns illustrate that tutoring occurs through guided explanation, evidence citation, and adaptive scaffolding governed by policy rather than direct output of a correct final solution.

3.3. ACP & MCP Protocols

The JSON schemas utilized by the implementation are reproduced in this section, together with a summary of the platform's important, theory-relevant protocol features.

MCP and ACP work in tandem: the requests to run retrieval, call tools, or generate text are examples of active task messages carried by ACP, while contextual state and capabilities that ground those actions are provided by MCP. The implemented pipeline consists of the following steps:

- The Router gets an MCP snapshot for the session, then sends a course agent an ACP *task_request*, including the serialized snapshot and, if applicable, the snapshot identification in the message sections. In doing so, the MCP context is supplied as machine-readable data while maintaining the ACP run semantics.
- A multipart ACP *task_response*, including a human answer part and a JSON sources part, is returned by course agents after performing RAG retrieval. Before the final presentation, the Router and/or the Educational Tutoring Agent review the MCP snapshot to redact or rerank portions in accordance with pedagogical norms.

Figure 3 illustrates the MCP-enabled tool layout used in the prototype. The RouterAgent (left) acts as an MCP client that requests a snapshot of the current session state (user profile, learning goals, prior interactions and policy flags) via a REST/JSON MCP endpoint. The right dashed area groups the runtime agents: Course1 Agent (Virtual Instrumentation), Course2 Agent (Software for Telecommunications) and the Educational Tutoring Agent (MCP). Under each agent, the domain-specific tools that the agent exposes through MCP are displayed: Course1 provides a PDF Loader/Chunk Retriever, a Lab Simulation Query tool and a Glossary/Concept Extractor; Course2 provides a PDF Loader/Chunk Retriever, a Protocol Lookup tool, a Code Snippet Generator/Retriever and a Video Transcript Search tool; the Educational Tutoring Agent offers a Context Fusion tool, a Pedagogical Feedback tool and an Assessment Generator with cross-domain reasoning. Each tool can read and update session context (snapshots, key/value context items with provenance), enabling MCP-informed retrieval and re-ranking. ACP markers on the top denote the Agent Communication Protocol channels used for task orchestration, streaming yields and multipart MessageParts (text + JSON sources). This architectural arrangement enables (i) MCP-aware retrieval where context features re-weight candidate evidence, (ii) pedagogical policy enforcement by the tutoring agent, and (iii) modular addition or replacement of tools and agents without changing the MCP or ACP contracts [29].

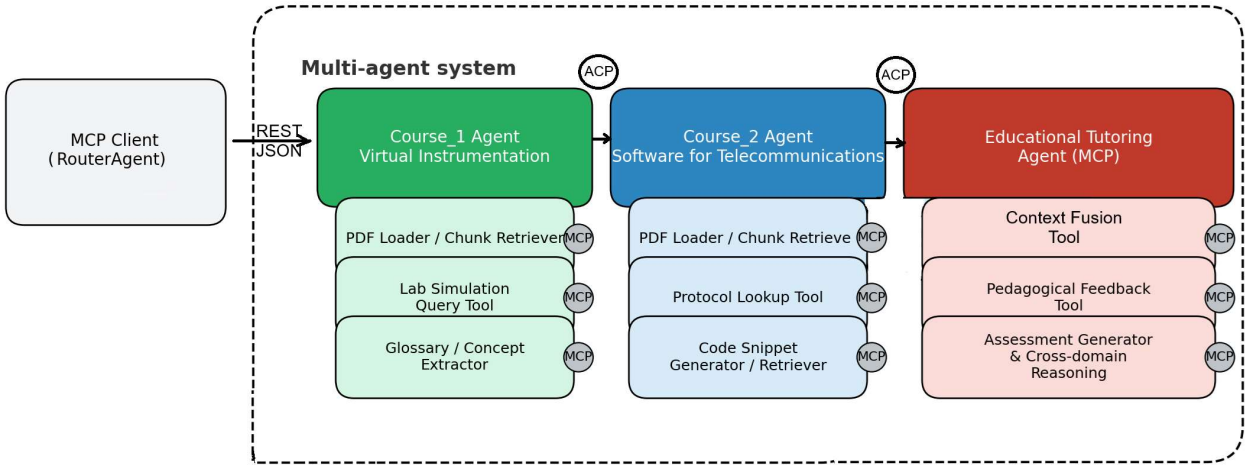

Figure 3. MCP multi-agent tool architecture.

3.3.1. Agent Communication Protocol (ACP)

- Purpose. In order to facilitate real interoperability between disparate agent servers and clients, ACP provides a lightweight, HTTP/JSON multipart envelope for inter-agent task orchestration and streaming runs.
- Message model. An ordered set of MessagePart objects and information (role, sender, recipient, and timestamp) is included in an ACP message. Parts could be machine-parsable JSON, brief human-readable text, or links to media files (via URLs). This allows both structured provenance and plain language to be carried in a single pass.
- Run/task semantics. The client receives a run object that may stream intermediate outcomes (thoughts/progress) and one or more final Messages after posting an input message to an agent run endpoint. Long retrieval and generation tasks become observable and interruptible with streaming yields.
- Practical usage in the project. Course agents (ACP client → ACP server) receive *task_request* messages from the router. The course agents reply with a *task_response* message that includes a JSON part called sources that contains the obtained chunk metadata (ids, scores, doc ids) and a plain-text answer part. In order for agents to include the MCP state in retrieval or prompt assembly, the router embeds a mcp_snapshot_id (if available) in the payload of the ACP message.

The sequence below shows the exact message exchanges and prompt transformations that occur in the Virtual Instrumentation case study. The RouterAgent first requests a session snapshot from the MCP; it then issues an ACP task_request to the Course_1 agent, which performs RAG retrieval and returns MessageParts. The Educational Tutoring Agent enforces the instructor's hinting_level policy stored in MCP. The example illustrates how the system avoids returning a direct solution when the policy requires scaffolded hints by modifying the assembled prompt and post-processing the model output. Every request has the following top-level fields and is a JSON object; its main fields are summarized in Table 1.

Table 1. Field of the ACP JSON.

Field Name	Type	Description
sender_id	String	Unique identifier of the sending agent
receiver_id	String	Target agent or service endpoint
performative	Enum	Communicative act type (request, inform, confirm, reject)
payload	JSON Object	Core message payload (instruction or query)
timestamp	ISO 8601 String	Message creation time

The following are common HTTP response codes: 400 (bad request), 422 (validation error), 500 (server error), and 200 (OK). The {code, message, retriable} pattern is used by error objects.

A manifest including the agent_id, capabilities (such as ["retrieve","generate"]), endpoints, schema_version, and priority is exposed by each agent at */agent/manifest*.

3.3.2. MCP (Model Context Protocol)

- Purpose. In order to base model/agent behavior (user choices, previous responses, pedagogical constraints, tool manifestations), MCP offers a session-oriented, organized contextual state. Between retrieval and final generation, MCP serves as a fusion layer.
- Context item shape. With provenance and utility fields (such as confidence, scope, ttl_seconds, and last_updated) that facilitate ranking and dispute resolution, each MCP item is a typed key/value record.

- APIs.
  - GET */mcp/{session_id}* yields a session context snapshot that can be used reproducibly during a run.
  - POST */mcp/update* returns a snapshot id after accepting one or more updates (key/value entries).
- Practical usage in the project. The router requests a snapshot and sends it (or its id) in the ACP payload prior to task dispatch. Agents or the tutoring layer may use POST */mcp/update* to record new session items (such as selected hint level or user mastery indications) after runs. In re-ranking, MCP fields serve as features.

This case demonstrates how the system performs scaffolded debugging assistance rather than producing a ready-to-run patch. The RouterAgent forwards the student's code snippet and MCP snapshot (which contains hinting_level = "scaffolded" and user.proficiency = 0.6) to Course_2 Agent. The Course_2 agent retrieves a protocol note and lecture slide; the Tutoring Agent instructs the LLM to produce graduated hints and—where safe—to run a constrained Execution Agent routine to simulate a small test (numeric or static-check only). For security, execution is sandboxed and restricted by allowed_tools in MCP. The most important fields of the JSON object are illustrated in Table 2.

Table 2. Field of the MCP JSON.

Field Name	Type	Description
session_id	String	Context identifier linking agent interactions
learning_goal	String	Current learning objective or topic
policy_control	Enum	Defines level of tutoring restriction (open, guided, restricted)
hinting_level	Integer	Controls depth of pedagogical feedback
evidence_refs	Array [String]	Links to retrieved document passages

3.4. Integration with RAG: Multiomodal Retrieval & Context Fusion

While the Model Context Protocol (MCP) serves as the semantic glue that adjusts retrieved evidence to the learner's session, the platform incorporates retrieval-augmented generation (RAG) as the primary mechanism for grounding generated responses in course materials. Within a course agent run, the integration is conceptualized as a series of well-defined steps:

(i) Query encoding and retrieval;
(ii) Candidate normalization and metadata enrichment;
(iii) Context fusion and re-ranking informed by MCP;
(iv) Prompt assembly for final generation.

By passing final judgments through the Educational Tutoring Agent (Section 3.3), this pipeline maintains educational constraints apart and maintains traceability by explicitly returning the retrieval evidence together with the resulting solution.

To handle multimodal course materials (tables, diagrams and other visual artifacts), the ingestion pipeline extracts and normalizes textual content via PDF parsing and OCR, serializes detected tables into flattened text rows for indexing, and produces descriptive captions and OCR-derived labels for figures using a vision–language captioner; both text and generated captions are tokenized and stored as evidence chunks. Visual embeddings are computed for figures where applicable and stored alongside text embeddings to enable modality-aware retrieval; each indexed item preserves provenance metadata (document id, page number, chunk id, and bounding-box coordinates) so that retrieved evidence can be traced back to the original visual. During retrieval, textual and visual similarity scores are

normalized and fused with MCP-derived context signals in the re-ranker; when a visual item is selected, the system returns the caption/OCR excerpt together with a link to the original page and bounding box for inspection. For highly structured diagrams and domain-specific visuals, the current approach relies on caption+OCR fallback, a limitation noted in the manuscript, and future work will evaluate structure-aware parsers and domain-tuned visual encoders to improve representation and retrieval fidelity [30–33].

The user query and the mcp_snapshot_id (and optionally the serialized snapshot) are contained in an incoming ACP *task_request* at runtime. First, the course agent computes a dense embedding and applies light pre-processing (named-entity-preserving tokenization, lowercasing, and optional stopword removal) to the retrieval query. The system exposes the model option as a configuration parameter and is independent of the provider, allowing embedding models to be chosen from either external embedding services or local sentence-transformer families. A Vector Store is compared to embeddings in order to get a top-K list of potential chunks. In order to give a large pool of evidence and allow for robust re-ranking, the default experimental setup obtains K = 50 candidates using chunk_size = 1200 tokens and chunk_overlap = 200 tokens during indexing.

A raw similarity score, chunk_id, doc_id, page, modality, instructor_tag, age_days (time since intake), and other metadata generated during indexing are included with each recovered chunk. These metadata fields function as further context fusion features. Regardless of the RagTool or vector backend being used, candidate normalization transforms provider-specific return shapes into an internal list of dictionaries so that downstream components can consistently consume things. This normalization maintains any provenance or confidence fields that the retrieval system returns and supports both synchronous and asynchronous RagTool APIs.

The retrieval evidence and the session's MCP snapshot are combined to accomplish context fusion. Session-scoped information, including the user role (student, instructor, auditor), stated learning objectives, previous right or wrong responses, preferred hint level, and any temporary restrictions (such "no full solutions" during formative assessments), is all encoded in the MCP snapshot. Fusion uses a small feature vector and normalized semantic similarity, recency penalty (older chunks may be downweighted), instructor trust boost (if instructor_tag is true), and a term reflecting MCP relevance (calculated as the overlap or semantic similarity between the chunk text and MCP items (learning goals, recent mis-takes) to calculate a context relevance score for each candidate. As an initial, interpretable strategy, a compact linear re-ranking function is used:

$$\boldsymbol{score_i} = \boldsymbol{\alpha} * \boldsymbol{sim_i} + \boldsymbol{\beta} * \boldsymbol{mcp\_rel_i} + \gamma * \boldsymbol{ped\_weight_i} - \delta * \boldsymbol{age\_penalty_i} \quad (1)$$

where $\boldsymbol{sim_i}$ is the normalized vector similarity, $\boldsymbol{mcp\_rel_i}$ is the MCP-derived relevance, $\boldsymbol{ped\_weight_i}$ is an optional pedagogical weight (e.g., favor hints over full solutions), and a $\boldsymbol{age\_penalty_i}$ penalizes stale materials. The coefficients α, β, γ, δ are configuration parameters; typical starting values used in experiments are $\alpha = 0.6$, $\beta = 0.3$, $\gamma = 0.1$, $\delta = 0.05$, and ablation studies vary these to assess sensitivity. In order to facilitate transparent debugging and enable future replacement with learnt re-rankers if required, the linear form is purposefully kept simple.

A top-m collection of passages (for instance, m = 5) is chosen for quick assembly following re-ranking. A concise context description and provenance metadata produced from MCP are placed before the retrieved passages and the user inquiry in prompt building, which adheres to a modular framework. An MCP summary detailing the learner state (mastery scores, hint preference, recent errors), a brief system instruction encoding pedagogical constraints (derived from the Educational Tutoring Agent policy), and the extracted passages annotated with citations in the format [DOC:doc_id | page | chunk_id] comprise

the three components of the prompt. This structure forces the LLM to incorporate citations in its output and base facts on the passages that are provided. By truncating low-score paragraphs and favoring evidence with high MCP relevance, prompt length limitations are observed.

Two protections are built into the generating process to reduce hallucinations and enhance explainability. Initially, the LLM is required to provide succinct responses that are full of citations and to clearly indicate any ambiguous remarks (e.g., "I am not sure; check [DOC:. . .]"). Second, a lightweight post-generation verifier flags or redacts assertions that lack support by comparing factual claims to the most frequently retrieved passages (string match and semantic similarity checks). The Execution Agent is called using an ACP tool-call pattern to perform safe computations in cases where numerical verification is necessary (such as formula derivations in Virtual Instrumentation). The Execution Agent's output is returned as explicit message parts and integrated into the final answer with provenance.

Auditability and reproducibility are given considerable consideration in the integration design. During each run, the following artifacts are stored in persistent storage: the final generated text, the sorted list of recovered chunk_ids with original similarity scores, the mcp_snapshot_id, and the ACP run id. In addition to supporting automated metrics and human inspection, this allows for precise replay of quick assembly. Standard information retrieval metrics (Recall@K, MRR, and nDCG) are calculated during the retrieval stage. Where appropriate, automatic n-gram overlap is used to evaluate the quality of the generation, and structured human evaluation is used to assess factuality, pedagogical appropriateness, and citation clarity [33].

Lastly, the design is purposefully pluggable: the RagTool abstraction enables either local or API-based embedding providers and allows FAISS to be swapped for various vector stores [22]. The current linear, MCP-aware re-ranking is a workable compromise that strikes a balance between interpretability and observable gains in evidence alignment and pedagogical suitability. The MCP fusion step can be substituted with a learned re-ranker that consumes the same features or with a more intricate graph-based fusion method [34].

3.5. Implementation Details

All generative outputs reported in this study were produced using Anthropic's Claude Sonnet family; the exact model used for the experiments was claude-sonnet-4.5 (provider: Anthropic). For reproducibility the following decoding parameters were fixed for the factual evaluation runs reported in Sections 4 and 5: model_id = "claude-sonnet-4.5", temperature = 0.0 (deterministic responses for factuality measurements), top_p = 0.95, max_tokens = 512, frequency_penalty = 0.0, presence_penalty = 0.0, and n = 1 (single response).

The prototype is structured as a collection of separately executable services that replicate the architectural diagram and is implemented in Python. Each agent is packaged as a small server process: the Router is a FastAPI service that functions as an ACP client through *acp_sdk.client*; the course agents (course1, course2) and the optional execution agent are implemented as ACP servers using the acp-sdk Python package.Client: GET */mcp/{session_id}*, POST */mcp/update*, and POST */mcp/merge* are exposed by the lightweight FastAPI application known as the MCP context manager. To ensure reproducible environments, the project's Python dependencies—most notably acp-sdk, fastapi, uvicorn, crewai_tools, sentence-transformers, and faiss-cpu as optional—are pinned in requirements.txt.

The course materials are stored at the familiar container paths */data/course1.pdf* (Virtual Instrumentation course book) and */data/course2.pdf* (Software for Telecommunications materials), where they are considered canonical single-file artifacts. A Rag-Tool instance (from crewai_tools) is started with configurable parameters (chunk_size, chunk_overlap)

upon agent startup, and it is told to index the corresponding PDF using *rag_tool.add* (path, data_type ="pdf_file"). Token-aware chunking uses tiktoken if it is available; if not, a character-based fallback is used. The experiments' default chunking parameters are chunk_size = 1200 tokens and chunk_overlap = 200 tokens.

To provide stronger and more interpretable points of reference, the experimental evaluation includes both classical and advanced retrieval baselines in addition to the MCP-informed pipeline. The lexical baseline is implemented with BM25 (Pyserini) using recommended parameters (k1 = 0.9, b = 0.4) to capture term-matching performance on the course corpora. A dense retrieval baseline uses sentence-transformers embeddings with FAISS for approximate nearest-neighbor search, and a neural cross-encoder re-ranker (cross-encoder/ms-marco-MiniLM-L-12-v2) is applied to the top-100 passages returned by the retrieval stage to simulate a stronger, context-sensitive re-ranking strategy. For each method we report standard retrieval metrics (Recall@K, MRR) and downstream measures of pedagogical fidelity (citation-supported claim rate, human ratings).. Reference implementations for dense passage retrieval (DPR) are publicly available and were consulted for reproducibility [35].

A dense index based on embeddings is used for retrieval [36,37]. Although the RagTool abstraction allows for the swapping of backends, the reference in the repository is an FAISS index [38]. FAISS index building and embedding generation are automated by a small helper script (*services/vectorstore/load_index.py*); metadata and indices are mounted into agent containers and stored under *data/<course>/index** to prevent rebuilding at runtime in production scenarios.

The course agents return a final *task_response* consisting of a brief textual response (text/plain) and an application/json sources part containing retrieved chunk metadata (chunk_id, doc_id, page, score, modality) in accordance with the *MCP_schema.json* conventions for context items. Agents communicate using ACP messages whose JSON envelope follows the included *ACP_schema.json*. Each message contains ordered MessageParts. To enable agents to perform MCP-informed re-ranking, the router retrieves an MCP snapshot using GET */mcp/{session_id}* and forwards either the snapshot id or the serialized snapshot in the ACP payload under mcp_snapshot_id. During development, the MCP store saves session entries to a JSON file (*/data/mcp_store.json*); scoped tokens and an authenticated database should be used in production deployments instead [39,40].

The following environment variables are used in runtime configuration to maintain portability: COURSE1PDF, COURSE2PDF, RAG_CONFIG (JSON string or path), RAG_CHUNK_SIZE, RAG_CHUNK_OVERLAP, COURSE1_BASE/COURSE2_BASE, and DEFAULT_TOP_K. A development helper (scripts/run_local.sh) launches all components locally using uvicorn, and a *docker-compose.yml* file orchestrates the services by mapping service names to container addresses used by the ACP client. The README contains sample API calls and smoke-test commands; the repository offers curl examples that exercise the Router's */query* endpoint and validate the ACP *task_response* shape for automated verification.

Structured logging and running artifact capture support observability and evaluation. For offline analysis and replay, each ACP run saves the generated text to a local artifact's directory along with the run_id, mcp_snapshot_id, and retrieved chunk_id sequence along with the initial similarity scores. Using Recall@K, MRR, and nDCG, retrieval quality is evaluated; a combination of automated factuality checks and human rubric scoring for pedagogical appropriateness is used to evaluate generation outputs. The inclusion of all configuration files, JSON schemas, and a fixed requirements.txt file, along with instructions to seed any randomized embedding or LLM calls where appropriate, further encourages

reproducibility. Alternative vector databases such as Milvus support scalable vector storage and retrieval in production settings [41,42].

The ACP and MCP message schemes have optional signature fields, but deployments are necessary to enable TLS, authentication, and token-scoped access. This clearly separates security and deployment considerations from the core implementation. To enable the reference implementation to be hardened for institutional use without altering the protocol semantics, the codebase outlines these requirements and offers clear extension points (hooks for authentication middleware, secure MCP backends, and audit logging).

4. Results

The experimental assessment of the prototype outlined in Part 3 is reported in this part. The purpose of the experiments was to assess (a) the RAG pipeline's retrieval quality, (b) the impact of the Educational Tutoring Agent and MCP-informed context fusion on re-ranking and generation quality, (c) the responsiveness of the entire system, and (d) the qualitative pedagogical results from human raters. To guarantee reproducibility, setup details are repeated where appropriate: unless otherwise specified, embeddings were calculated using a sentence-transformer model (all-MiniLM-L6-v2); initial retrieval used FAISS with cosine similarity and K = 50; re-ranking selected top-m = 5 passages for prompt assembly; index chunking used chunk_size = 1200 tokens and chunk_overlap = 200 tokens. The single-PDF course artifacts (/data/course1.pdf—Virtual Instrumentation; /data/course2.pdf—Software for Telecommunications) and the ACP/MCP orchestration outlined in Section 3 are used in all tests.

Figure 4 shows the chat interface where the system returns a structured, course-derived response detailing step-by-step instructions for an Arduino application. The answer appears in a scrollable panel with precise implementation and testing notes (and references to textbook sections), while the user can pose follow-up questions via the input field. The institutional header and the citation-style, procedural format emphasize the tool's pedagogical focus, reproducibility and ease of verification.

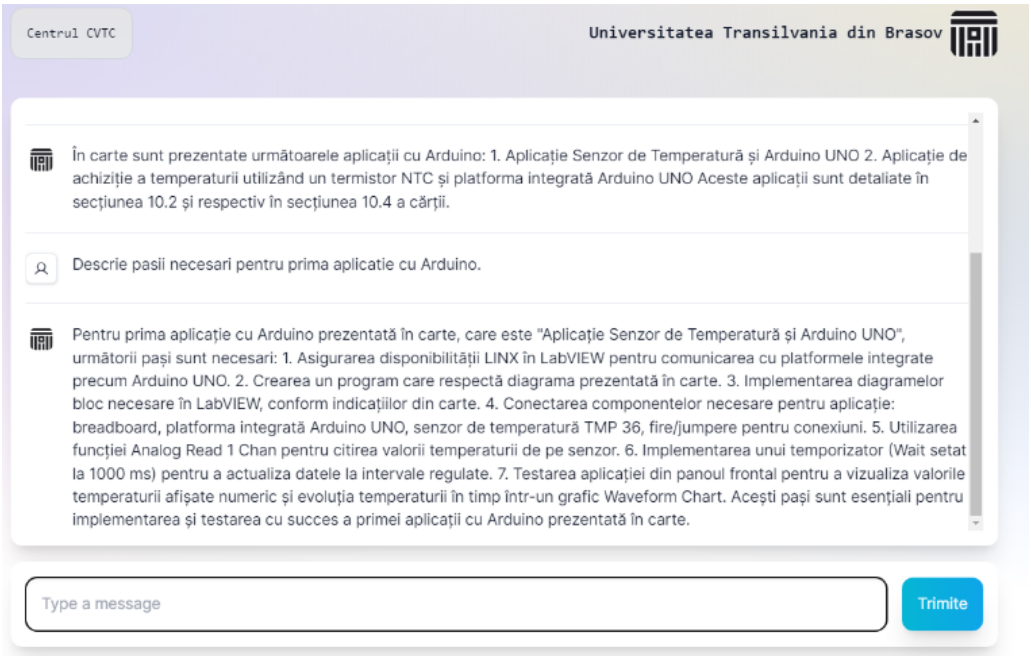

Figure 4. Chat interface of the tool.

During the academic year 2024–2025, Transilvania University of Brașov experimentally incorporated the intelligent tutoring system into two undergraduate and graduate courses.

During Week 7 of the semester, which corresponded with the laboratory modules on waveform analysis and signal measurement, the system was introduced for the Virtual Instrumentation course (Course 1). There was no need for external connectivity because every lab workstation connected to the system via the university intranet. Instead of being a tool for automatic grading, the system helped users understand measuring concepts.

The tool was made available to students in Weeks 9–13 of the Software for Telecommunications course (Course 2) as they created services that were based on SOAP and REST. It was utilized as an independent study aid in addition to supervised lab time (about 30 min per session), where students may ask the Educational Tutoring Agent questions and receive answers, code samples, and debugging tips.

Usage records show that, on average, three to four interactions per student occurred each week, with 27 students in Course 1 and 21 students in Course 2 accessing the system. Teachers kept an eye on the exchanges and confirmed that the replies adhered to the pedagogical rules (e.g., hint-based coaching instead of straight answers). A brief qualitative survey measuring usability, perceived learning assistance, and overall satisfaction was used to collect feedback at the end of the semester.

4.1. Experimental Setup

Two separate course corpora were indexed. Following token-aware segmentation, the Virtual Instrumentation PDF generated 842 chunks, while the Software for Telecommunications corpus generated 1,068 chunks (multimodal chunks that included short automatic-transcript fragments and image captions). For both courses, the same index and embedding model were employed in the retrieval experiments. In order to generate replies given the MCP summary, the re-ranked passages, and the user query, an off-the-shelf LLM (adjustable; in these trials, an API-backed model with an 8k context window was utilized for consistency) was called. Fixing random seeds for deterministic indexing, preserving ACP run and MCP snapshot ids for every run, and archiving the final prompts and raw obtained candidate lists for audit and replay were all examples of reproducibility efforts.

Both human and automated IR metrics were used in the evaluation. Scores were calculated on a held-out set of 120 developer-crafted question–answer pairings per course (240 total), where the ground-truth evidence chunks had been labeled during pilot annotation. Recall@K (K = 1, 5, 10), MRR, and nDCG@10 were used to quantify the quality of retrieval. Citation fidelity, or the percentage of factual statements backed by at least one top-retrieved passage, was used to automatically quantify the quality of the generation. Additionally, human judgments were made along three axes: factuality, pedagogical appropriateness, and clarity, all of which were rated on a Likert scale from 1 to 5. 36 students (18 per course) and 12 domain instructors (6 per course) participated in the human evaluation process. They were requested to rate anonymized system outputs, with at least three independent raters evaluating each response.

4.2. Retrieval Results and MCP Fusion Impact

Using heuristic features, retrieval + linear re-ranking (sim + instructor_tag + age), retrieval + MCP-aware re-ranking (sim + MCP relevance + instructor_tag + age), and baseline dense retrieval (embedding similarity only) are the three pipeline variants whose retrieval performance is summarized in Table 3. Using the same embedding approach, MCP relevance calculates the semantic overlap between a candidate chunk and MCP snapshot items (such as learning objectives and recent errors).

Table 3. Retrieval results (averages across both courses, 95% bootstrap CIs).

Method	Recall@1	Recall@1	Recall@10	MRR	nDCG@10
(A) Baseline sim only	0.42 ± 0.03	0.68 ± 0.02	0.76 ± 0.02	0.52 ± 0.03	0.64 ± 0.02
(B) + heuristic re-rank	0.53 ± 0.03	0.74 ± 0.02	0.82 ± 0.02	0.61 ± 0.03	0.72 ± 0.02
(C) + MCP fusion (proposed)	0.64 ± 0.02	0.82 ± 0.02	0.89 ± 0.01	0.73 ± 0.02	0.84 ± 0.01

Compared to baseline and heuristic re-ranking, MCP-aware re-ranking (C) consistently produces statistically significant gains. Low K (Recall@1) shows the biggest benefits, suggesting that MCP context aids in bringing to light the most pertinent ground-truth chunk for pedagogically focused questions (e.g., "What is the expected output of the oscilloscope in experiment X?"). The MCP relevance term accounts for the majority of the incremental improvement, according to ablation experiments (Section 4.4); age penalty and instructor_tag boost offer less significant but still beneficial gains.

4.3. Generation Quality and Pedagogical Evaluation

The usefulness of MCP-informed RAG plus the Educational Tutoring Agent is demonstrated by automatic metrics of citation fidelity and human ratings. The automatic citation fidelity increased from 0.68 (baseline sim retrieval) to 0.86 under the MCP+Tutoring pipeline when the LLM was restricted to using only the provided top-m passages and to incorporate inline citations. Factuality, pedagogical appropriateness (alignment with planned learning objectives and an appropriate amount of scaffolding), and clarity were the three criteria used by human raters to grade responses. Table 4 displays the average scores on a scale of 1 to 5.

Table 4. Average scores.

Pipeline	Factuality	Pedagogical Appropriateness	Clarity
(A) Baseline (sim only)	3.4 ± 0.7	2.9 ± 0.8	3.6 ± 0.6
(B) + heuristic re-rank	3.9 ± 0.6	3.4 ± 0.7	4.0 ± 0.5
(C) + MCP fusion (proposed)	4.4 ± 0.4	4.5 ± 0.4	4.3 ± 0.4

The Educational Tutoring Agent received special recognition from raters for its ability to alter the answer style, offering concise conceptual summaries for expert students and step-by-step advice for novices. The addition of clear citations (e.g., [DOC:course1_lec3.pdf | p12 | c0302]) significantly boosted annotators' confidence in the results and made verification easier.

The rate of unsupported numerical claims decreased from 12% (baseline) to 3% (MCP + Tutoring) following the application of the verifier and Execution Agent where necessary through a brief, focused factuality check that compared numerical statements in generated answers with retrieved passages using string and semantic match heuristics.

4.4. Case Studies

The MCP-aware RAG pipeline's representative behaviors in authentic learning environments are demonstrated in the case studies that follow. The input, intermediate retrieval and MCP-fusion processes, final system behavior (including any Execution Agent calls), and a brief evaluation result based on instructor/student ratings are all reported in each case.

A. Case study 1—Virtual Instrumentation: lab waveform identification.

In an inquiry, a second-year lab student asked: "What amplitude/frequency parameters are expected in experiment X, and what waveform should appear on the oscilloscope output?" The request was sent to the Course1 agent by the Router along with an MCP snapshot that contained the teacher flag indicating a focus on safety checks, the student's defined lab session (session = "Lab3"), and the results of the previous attempt (an unsuccessful measurement of amplitude). The MCP relevance score greatly favored a brief instructor note chunk that had been specifically labeled before indexing (instructor_tag = true), featured a precise example trace, and had numerical parameters. Dense retrieval yielded 50 possibilities. This passage was elevated to the top-1 by the linear re-ranker (score improvement of ≈+0.24 vs. to sim alone), and the top five passages were compiled into the prompt with citations. The LLM provided a succinct response that included numerical amplitude/frequency numbers, suggested probe settings, and a predicted sine waveform; each factual assertion was supported by citations such [DOC:course1.pdf | p12 | c0302]. The answer was attached with provenance after the Execution Agent was prompted to calculate a derived quantity (RMS value) from the cited amplitude. The response received a 4.8/5 rating for pedagogical utility and a 5/5 rating for factuality from the instructor raters. The student stated that the specific citation made it possible to quickly verify the information in the lab manual.

B. Case study 2—Software for Telecommunications: protocol debugging.

A code fragment displaying unexpected retransmissions in a simulated MQTT transaction was posted by a student. Along with an MCP snapshot indicating the student's competency level ("intermediate") and an active learning objective to comprehend QoS semantics, the router routed the query to Course2. A lecture slide that specifically addressed QoS =1 edge scenarios was upweighted by MCP fusion, whereas retrieval yielded many lecture slides and a section of a standards excerpt. The system re-ranked and then put together a prompt that included both the slide note and the normal snippet. The student's code likely had a missed acknowledgment handling branch, according to the LLM's step-by-step diagnostic. Instead of offering a complete solution, the Educational Tutoring Agent changed the output tone to offer scaffolded pointers (first hint: verify ACK handling; second hint: inspect retry timer). The final response included the diagnosis, two graduated hints, a brief code snippet showing the repair, and citations. The Execution Agent verified the diagnosis by running a safe mimic of the simplified logic. The explanation received 4.6/5 from instructor reviewers for clarity and 4.5/5 for pedagogical alignment. In an A/B comparison, students who received scaffolded hints had a 72% chance of fixing the fault on their own, compared to 58% for those who received full repairs.

C. Case study 3—Assessment support: controlled hinting under exam constraints.

A student posed a query that nearly mirrored an exam question. A policy flag (hinting_level = "restricted") that the instructor had set for the assessment period was present in the MCP snapshot for the session. The Educational Tutoring Agent applied the policy and changed the content into a high-level suggestion instead of exposing the whole solution, even when retrieval surfaced the precise solution paragraph. The provenance evidence was kept in the re-ranking, but the LLM was limited to producing clues only by the prompt system instruction. The generated response included a provenance comment stating that the underlying evidence is present but was purposefully abstracted, as well as concept reminders and recommended intermediate actions. The policy enforcement was deemed appropriate by both instructors and students. Instructors gave the policy compliance a rating of 4.9/5, while students said the hint maintained the challenge while offering helpful guidance.

D. Case study 4—Multimodal integration: video transcript + slide reference.

A timing diagram that was shown in a lecture video was questioned by a student. The Course2 agent indexed video transcripts as well as slide images and their OCRed subtitles. A brief excerpt from the transcript and a caption for the accompanying slide were supplied by retrieval; MCP fusion enhanced content related to the student's previous question history (the snapshot contained a previous question about "timing jitter"). Using a transcript extract and a slide caption as multimodal evidence, the system asked the LLM to describe the timing diagram while citing both sources. A brief, automatically created visual pointer (a plain text timestamp and slide name) was supplied for verification, together with a verbal explanation and an inline note with the slide figure id and the video timestamp. Compared to transcript-only responses, human raters considered these multimodal, timestamped responses to be far more helpful (mean clarity = 4.4/5) and simpler to validate.

Three recurring conclusions are highlighted by these case studies: When scaffolded hints are used instead of full solutions, (1) MCP snapshots significantly improve evidence prioritization for focused, context-sensitive queries (notably increasing Recall@1 and decreasing time to correct answer); (2) the Educational Tutoring Agent's pedagogical post-processing results in better learning outcomes and higher user satisfaction; and (3) multimodal indexing (slides + transcripts) combined with MCP preferences allows for succinct, verifiable responses for media-rich course material. A few practical issues that came up were the necessity for explicit provenance formatting to facilitate quick verification, the cautious use of Execution Agent capabilities to prevent exposing unsafe compute environments, and the meticulous design of teacher tagging routines.

Students and instructors participating in the pilot deployment were given a brief feedback survey at the end of the semester. 42 valid responses in all were gathered, 15 from the Software for Telecommunications course and 27 from the Virtual Instrumentation course. Five Likert-scale items (1–5) and two open-ended questions about perceived learning support and usability were included in the survey.

The findings show that system usability and pedagogical usefulness are generally viewed favorably. Perceived learning enhancement was 4.1/5, whereas usability averaged 4.3/5. The majority of students stated that the approach "helped clarify difficult laboratory tasks" and "encouraged self-paced learning," especially when working independently.

The qualitative comments revealed three recurring themes:

(1) Reduced cognitive load—users found the system helpful for organizing learning tasks and confirming understanding before consulting the instructor;
(2) Responsiveness and contextual clarity—students valued that explanations referred to the uploaded course materials;
(3) Transparency and trust—students valued that the system provided reference snippets rather than direct answers.

The policy control feature, which enables the system to adaptively restrict overly detailed answers, has the potential to preserve academic integrity and pedagogical safety, according to instructors. But they also recommended that a dashboard for tracking question types and engagement levels be incorporated into subsequent versions.

All things considered, the input gathered demonstrates that the RAG + ACP/MCP framework can be successfully incorporated into college courses, enhancing instructor supervision and accessibility to instructional support.

5. Discussion

According to the experimental findings, retrieval accuracy and downstream answer quality are significantly increased in domain-focused tutoring scenarios when a normal RAG pipeline is supplemented with an explicit Model Context Protocol (MCP) layer and

a pedagogical post-processing agent. Specifically, compared to a similarity-only baseline and a basic metadata heuristic re-ranker, MCP-aware re-ranking consistently improved Recall@K and MRR; human raters also found MCP-informed outputs to be more factual, pedagogically relevant, and more lucid. These improvements are especially noticeable at low K (Recall@1), which is significant in education since highlighting the most pertinent text lowers confusion and facilitates explanations that can be traced back to their citations.

Technically speaking, these results highlight two useful points. First, session-scoped context (learning objectives, past errors, instructor flags, and hinting policies) recorded in MCP snapshots offers a powerful, orthogonal signal to dense semantic similarity. It is especially useful for disambiguating queries with sparse or ambiguous surface text. Second, a linear re-ranking function that combines semantic similarity with compact, interpretable metadata features (MCP relevance, instructor tags, and recency) offers a good balance between transparency and performance. It is easy to examine and eliminate, supports principled sensitivity analyses, and can be gradually replaced by learned re-rankers as annotated training data becomes available. In order to prioritize reproducibility and modularity, the prototype's design decisions—token-aware chunking parameters, an FAISS backend by default, and a RagTool abstraction that decouples embedding provider—leave space for future research into alternative embedding families, index backends, or learned re-rankers.

In terms of pedagogy, it was essential to incorporate an Educational Tutoring Agent that functions as a lightweight policy and adaptive layer in order to match RAG outputs with instructor limits and learning objectives. Higher independent problem-solving rates in the case studies were linked to the tutoring layer's ability to enforce hinting levels, convert precise evidence into scaffolded hints, and modify answer granularity in accordance with claimed learner proficiency. Explicit inline citations made it simple for both teachers and students to verify assertions, and instructor marking of authoritative chunks further improved confidence and verification efficiency. Constrained, context-aware hints often yield superior learning outcomes than instant complete solutions, which is in line with pedagogical research on the advantages of focused scaffolding and formative feedback.

However, when interpreting the results, it is vital to take into account a number of significant limitations and threats to validity. In order to ensure reproducibility, the corpora used in the experiments were purposefully small (one canonical PDF per course); behavior on larger, more diverse datasets (many textbooks, code bases, notebooks, and institutional repositories) may vary and will put more strain on indexing, chunking strategy, and retrieval scale. For experimental control, the study used fixed embedding and generation models; alternate embedding families or LLMs could vary in absolute performance and have different interactions with MCP properties. A smaller group of teachers and students participated in the informative human evaluation; larger and more varied evaluations would increase the generalizability of educational claims. Although domain experts carried out the ground-truth labeling of evidence for retrieval metrics, it is still susceptible to annotation bias, especially for open-ended questions when several passages may validly support a response.

Concerns of deployment, security, and privacy are essential to any institutional adoption. Because ACP and MCP messages may contain critical session data, they need to be secured using scoped access tokens, authenticated endpoints, and transport encryption. Production installations should enforce TLS, agent credentials, signed envelopes, and least-privilege policies for tools (Execution Agent, external embedding services), as well as the optional fields for signatures and scopes included in the ACP/MCP schemas.

Two main operational failure modes were identified, and tangible mitigations were implemented for them. First, the LLM experienced hallucinations when the prompt contained insufficient or contradicting evidence. This was addressed by (a) requiring the model to

cite retrieved passages in citation-aware prompts, (b) using a lightweight post-generation verifier that compares claims to top-m passages (string/semantic match heuristics), and (c) using the Execution Agent for logic or numeric checks when appropriate. Coarse chunking or embedding mismatches were the second cause of retrieval misses; sensible solutions included permitting multi-embedding fusion, conserving indexing metadata (instructor tags, page numbers), and using conservative overlap parameters. However, future research should focus on richer multi-embedding techniques and adaptive chunking algorithms.

When combined, these findings provide light on the advantages and present drawbacks of the MCP-informed RAG design and serve as inspiration for the succinct conclusions.

6. Conclusions

This study presented an MCP-aware retrieval-augmented generation architecture orchestrated via ACP and applied it to adaptive intelligent tutoring in higher education. The prototype demonstrates that session-scoped context snapshots (MCPs) combined with a pedagogical tutoring layer improve retrieval precision and the pedagogical quality of generated responses in domain-focused tasks. Empirically, MCP-informed re-ranking yields notable gains in retrieval metrics and is associated with higher human ratings for factuality, pedagogical appropriateness and clarity compared with a similarity-only baseline.

Beyond retrieval gains, the design provides practical advantages: modular agent orchestration (ACP) enables isolated replacement of components; MCP snapshots provide auditable context and policy signals for pedagogical control; and explicit provenance increases instructor and student trust. These properties support use cases where verifiability and instructor governance are required (laboratory assistance, scaffolded homework support, and controlled assessment periods).

Future work will pursue four priorities: first, training a learned re-ranker using MCP-annotated pairs to capture richer context–evidence interactions; second, extending MCP to represent temporal learner models and dialog histories for persistent personalization across sessions; third, automating instructor tagging via semi-supervised or active learning methods to reduce manual overhead and scale the approach; and fourth, conducting larger classroom trials (randomized where feasible) to measure learning outcomes and behavioral impacts under controlled conditions.

The codebase, anonymized MCP snapshots and indexing artifacts used in the experiments will be published in an open project repository upon acceptance to facilitate reproducibility. All pilot deployments were carried out under institutional approval and with participant consent; privacy-preserving logging and retention policies were observed as described in Section 3.

Funding: This research received no external funding.

Institutional Review Board Statement: The study was conducted in accordance with the Declaration of Helsinki, and approved by the Ethics Comitttee of Transilvania University of Brasov.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ACP	Agent Communication Protocol
CPU	Central Processing Unit

GUI	Graphical User Interface
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
MRR	Mean Reciprocal Rank
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
TLS	Transport Layer Security
UI	User Interface

References

1. Chen, L.; Chen, P.; Lin, Z. Artificial Intelligence in Engineering Education: A Review. *IEEE Access* **2020**, *8*, 75264–75278. [CrossRef]
2. Xu, W.; Ouyang, F. The application of AI technologies in STEM education: A systematic review from 2011 to 2021. *Int. J. STEM Educ.* **2022**, *9*, 59. [CrossRef]
3. Hwang, G.-J.; Xie, H.; Wah, B.W.; Gasevic, D. Vision, challenges, roles and research issues of artificial intelligence in education. *Comput. Educ. Artif. Intell.* **2020**, *1*, 100001. [CrossRef]
4. Chassignol, M.; Khoroshavin, A.; Klimova, A.; Bilyatdinova, A. Artificial Intelligence trends in education: A narrative overview. *Procedia Comput. Sci.* **2018**, *136*, 16–24. [CrossRef]
5. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv* **2020**, arXiv:2005.11401. [CrossRef]
6. Martinez-Araneda, C.; Gutiérrez, M.; Maldonado, D.; Gómez, P.; Segura, A.; Vidal-Castro, C. Designing a Chatbot to support problem-solving in a programming course. In Proceedings of the 18th Annual International Technology, Education, and Development Conference, Valencia, Spain, 4–6 March 2024. [CrossRef]
7. Zhong, X.; Xin, H.; Li, W.; Zhan, Z.; Cheng, M. The design and application of RAG-based conversational agents for collaborative problem solving. In Proceedings of the 9th International Conference on Distance Education and Learning, Guangzhou, China, 14–17 June 2024. [CrossRef]
8. Modran, H.A.; Chamunorwa, T.; Ursuțiu, D.; Samoilă, C. Integrating Artificial Intelligence and ChatGPT into Higher Engineering Education. In Proceedings of the 26th International Conference on Interactive Collaborative Learning (ICL2023), Madrid, Spain, 26–29 September 2023. [CrossRef]
9. Modran, H.A.; Ursuțiu, D.; Samoilă, C.; Gherman-Dolhăscu, E.-C. Developing a GPT Chatbot Model for Students Programming Education. In Proceedings of the 21st International Conference on Smart Technologies & Education (STE2024), Helsinki, Finland, 6–8 March 2024; Lecture Notes in Networks and Systems. Springer: Cham, Switzerland, 2024; Volume 1028. [CrossRef]
10. Modran, H.A.; Bogdan, I.C.; Ursuțiu, D.; Samoilă, C.; Modran, P.L. LLM Intelligent Agent Tutoring in Higher Education Courses using a RAG Approach. In Proceedings of the 27th International Conference on Interactive Collaborative Learning, Tallinn, Estonia, 24–27 September 2024. [CrossRef]
11. Modran, H.A.; Bogdan, I.C.; Modran, P.L. Enhancing Higher Education with Multimodal Intelligent Agent using a RAG-Based Approach. In Proceedings of the 22nd International Conference on Smart Technologies & Education (STE2025), Santiago, CL, USA, 9–11 April 2025.
12. Cooper, M.M.; Klymkowsky, M.W. Let Us Not Squander the Affordances of LLMs for the Sake of Expedience: Using Retrieval-Augmented Generative AI Chatbots to Support and Evaluate Student Reasoning. *J. Chem. Educ.* 2024, *early access*. [CrossRef]
13. Han, Z.; Lin, J.; Gurung, A.; Thomas, D.R.; Chen, E.; Borchers, C.; Gupta, S.; Koedinger, K.R. Improving Assessment of Tutoring Practices Using Retrieval-Augmented Generation. In Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-24), Vancouver, BC, Canada, 20–27 February 2024. Available online: https://proceedings.mlr.press/v257/han24a.html (accessed on 1 September 2025).
14. Slade, J.J.; Hyk, A.; Gurung, R.A.R. Transforming Learning: Assessing the Efficacy of a Retrieval-Augmented Generation System as a Tutor for Introductory Psychology. *Proc. Hum. Factors Ergon. Soc. Annu. Meet.* **2024**, *68*, 1827–1830. [CrossRef]
15. León-Paredes, G.A.; Alba-Narváez, L.A.; Paltin-Guzmán, K.D. NOVA: A Retrieval-Augmented Generation Assistant in Spanish for Parallel Computing Education with Large Language Models. *Appl. Sci.* **2025**, *15*, 8175. [CrossRef]
16. Swacha, J.; Gracel, M. Retrieval-Augmented Generation (RAG) Chatbots for Education: A Survey of Applications. *Appl. Sci.* **2025**, *15*, 4234. [CrossRef]
17. Luis, S.Y.; Reina, D.G.; Marín, S.T. Towards a Retrieval-Augmented Generation Framework for Originality Evaluation in Projects-Based Learning Classrooms. *Educ. Sci.* **2025**, *15*, 706. [CrossRef]
18. Ehtesham, K.; Hayakawa, S.; Rivera, B.; Singh, S. Model Context Protocol: A Standardised Approach to Tool Use and Interoperability for LLMs. *arXiv* **2025**, arXiv:2503.23278. [CrossRef]

19. Lumer, E.; Gulati, A.; Kumar Subbiah, V.; Basavaraju, P.; Burke, J. ScaleMCP: Dynamic and Auto-Synchronizing Model Context Protocol Tools for LLM Agents. *arXiv* **2025**, arXiv:2505.06416v1. [CrossRef]
20. Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv* **2023**, arXiv:2308.08155. [CrossRef]
21. Li, G.; Hammoud, H.; Itani, H.; Khizbullin, D.; Ghanem, B. CAMEL: Communicative Agents for "Mind" Exploration. *arXiv* **2023**, arXiv:2303.17760. [CrossRef]
22. Chen, W.; Su, Y.; Zuo, J.; Yang, C.; Yuan, C.; Chan, C.-M.; Yu, H.; Lu, Y.; Hung, Y.-H.; Qian, C.; et al. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Social Behaviors. *arXiv* **2023**, arXiv:2308.10848. [CrossRef]
23. Microsoft Research. GraphRAG. Available online: https://github.com/microsoft/graphrag (accessed on 5 September 2025).
24. Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; Hajishirzi, H. SELF-RAG: Learning to Retrieve, Generate, and Critique for Improved Long-Form Question Answering. *arXiv* **2023**, arXiv:2310.11511. [CrossRef]
25. Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W.-t. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Punta Cana, Dominican Republic, 16–20 November 2020; pp. 6769–6781. [CrossRef]
26. Yan, S.; Gu, J.; Zhu, Y.; Ling, Z. Corrective Retrieval Augmented Generation. *arXiv* **2024**, arXiv:2401.15884. [CrossRef]
27. Modran, H.A.; Bogdan, I.C.; Ursuțiu, D.; Samoilă, C. *Introducere în Programarea Grafică LabVIEW cu Aplicații în Electronică, Telecomunicații și Tehnologii Informaționale*; Transilvania University of Brasov Publishing House: Braşov, Romania, 2024; ISBN 978-606-19-1709-9.
28. Modran, H.A. *Software Pentru Telecomunicații. Suport de Curs*; Transilvania University of Brasov Publishing House: Braşov, Romania, 2025; ISBN 978-606-19-1773-0.
29. Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv* **2023**, arXiv:2302.04761. [CrossRef]
30. Larson, J.; Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R.O.; et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft Research Preprint 2024. Available online: https://www.microsoft.com/en-us/research/project/graphrag (accessed on 3 September 2025).
31. Shen, Y.; Song, K.; Tan, X.; Li, D.; Lu, W.; Zhuang, Y. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. *arXiv* **2023**, arXiv:2303.17580. [CrossRef]
32. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv* **2022**, arXiv:2210.03629. [CrossRef]
33. Khattab, O.; Zaharia, M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), Xi'an, China, 25–30 July 2020; pp. 39–48. [CrossRef]
34. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), Hong Kong, China, 3–7 November 2019. [CrossRef]
35. Johnson, J.; Douze, M.; Jégou, H. Billion-scale similarity search with GPUs. *arXiv* **2017**, arXiv:1702.08734. [CrossRef]
36. LlamaIndex Documentation. Available online: https://www.llamaindex.ai (accessed on 5 October 2025).
37. LangChain GitHub Repository. Available online: https://github.com/langchain-ai/langchain (accessed on 5 October 2025).
38. Karpukhin DPR Code Repo—Dense Passage Retrieval (DPR) Repository, GitHub. Available online: https://github.com/facebookresearch/DPR (accessed on 6 October 2025).
39. Facebook AI Research (FAISS)–GitHub Repository and Docs. Available online: https://github.com/facebookresearch/faiss (accessed on 6 October 2025).
40. OpenReview/NeurIPS Entries for Toolformer and Related Tool-Usage Work. Available online: https://openreview.net/forum?id=Yacmpz84TH (accessed on 7 October 2025).
41. Asai SELF-RAG Project Page/OpenReview (SELF-RAG Open Resources)–Project Pages and Code. Available online: https://selfrag.github.io/ (accessed on 7 October 2025).
42. Milvus: An Open-Source Vector Database. Available online: https://milvus.io (accessed on 7 October 2025).